



*Université Polytechnique de
Bingerville*

**MASTER 1
DATA-IA
MODULE: BIG DATA**

RAPPORT DE DÉPLOIEMENT ET D'EXPLOITATION D'UN CLUSTER AMAZON EMR

Réalisé par :

YAO MIÉZAN SAM WILLIAM

Enseignant :

Dr BOBET GOUALO

Rapport de déploiement et d'exploitation d'un Cluster Amazon EMR

SOMMAIRE

1. Introduction	1
2. Déploiement du cluster Amazon EMR	1
2.1 Préparation de l'environnement AWS	1
2.2 Configuration de la connexion SSH via PuTTY	2
2.3 Création et démarrage du cluster EMR	3
2.4 Configuration réseau et sécurité	5
2.5 Connexion au nœud maître	5
3. Exploitation du cluster EMR	6
3.1 Vérification des services Hadoop, YARN et Spark	6
3.2 Architecture du Data Lake sur HDFS	6
3.3 Ingestion des données depuis Amazon S3	7
4. Traitement ETL avec Spark	7
4.1 Script de conversion CSV vers Parquet	7
4.2 Vérification des résultats de l'ETL	8
5. Analyses Spark SQL	9
5.1 Script d'analyse métier	9
5.2 Résultats obtenus	10
6. Supervision et preuve du parallélisme distribué	11
6.1 Journaux d'exécution Spark	11
6.2 Vérification finale de la zone Curated	11
6.3 Interfaces de supervision du cluster	12
7. Comparaison EC2 manuel vs EMR managé	17
8. Conclusion	17

Rapport de déploiement et d'exploitation d'un Cluster Amazon EMR

1. Introduction

Dans la continuité du travail réalisé sur l'installation manuelle de Hadoop/Spark en mode pseudo-distribué sur une instance EC2 unique, cette partie du rapport présente le déploiement et l'exploitation d'un cluster Amazon EMR (Elastic MapReduce) multi-nœuds entièrement géré par AWS. L'objectif est de reproduire le même pipeline de traitement (ingestion, transformation CSV vers Parquet, analyses Spark SQL) sur une infrastructure Big Data managée et distribuée, puis de comparer les deux approches.

Le dataset utilisé reste celui des 100 000 transactions mobile money, couvrant quatre opérateurs ivoiriens : MTN CI, Orange Money, Moov Africa et Wave.

2. Déploiement du cluster Amazon EMR

2.1 Préparation de l'environnement AWS

Avant la création du cluster, deux ressources préalables ont été mises en place :

- Un compartiment Amazon S3 dédié à la centralisation des journaux (logs) du cluster EMR.
- Une paire de clés SSH (format .ppk pour PuTTY) permettant l'authentification sécurisée sur le nœud maître.

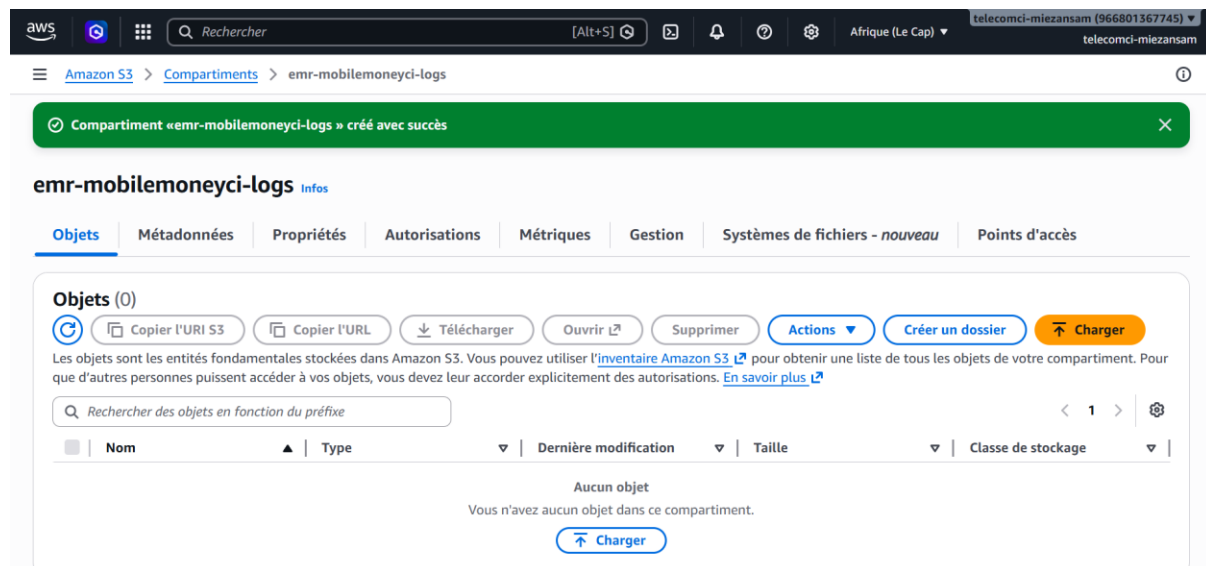


Figure 1 : Création du compartiment S3 « *emr-mobilemoneyci-logs* » pour les journaux du cluster

Rapport de déploiement et d'exploitation d'un Cluster Amazon EMR

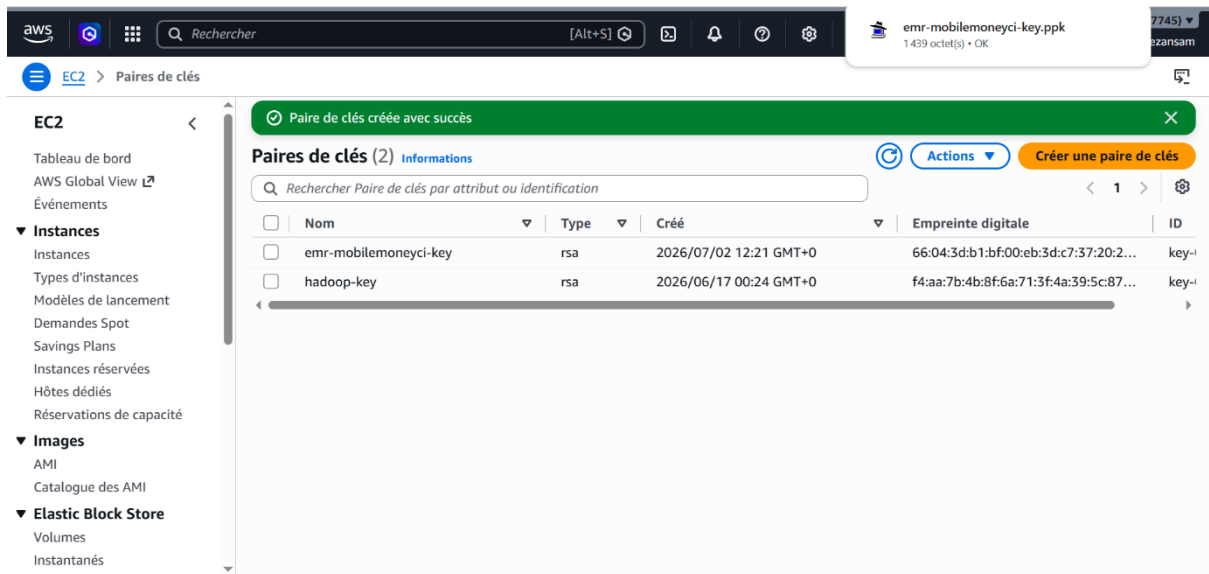


Figure 2 : Création de la paire de clés SSH « *emr-mobilemoneyci-key* » (EC2 > Paires de clés)

2.2 Configuration de la connexion SSH via PuTTY

La clé privée générée a été configurée dans PuTTY afin de permettre la connexion au nœud maître du cluster une fois celui-ci démarré. La session a été enregistrée sous le nom « *emr-cluster* » pour faciliter les connexions ultérieures.

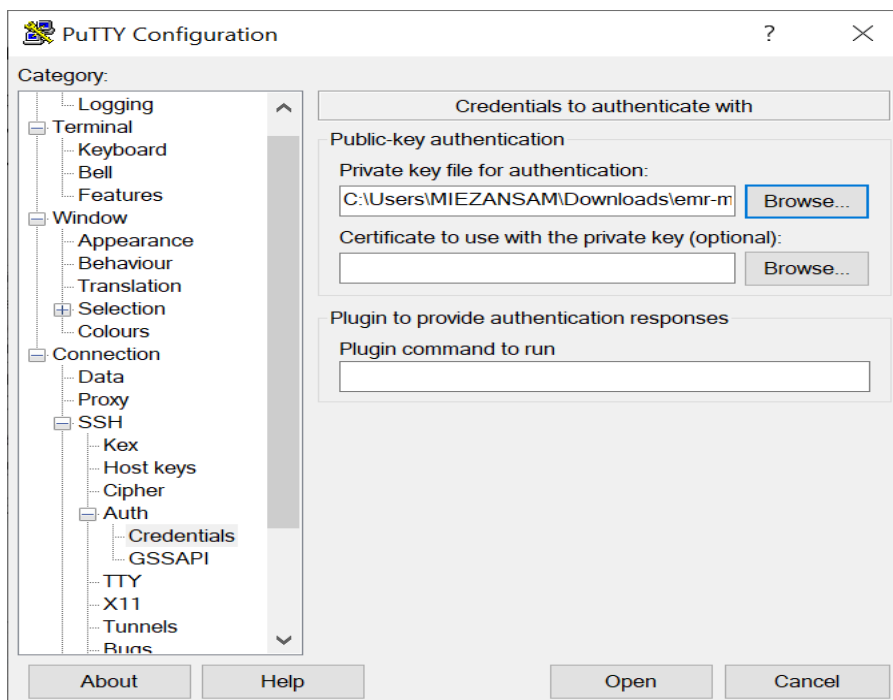


Figure 3 : Configuration de la clé privée d'authentification dans PuTTY (Connection > SSH > Auth > Credentials)

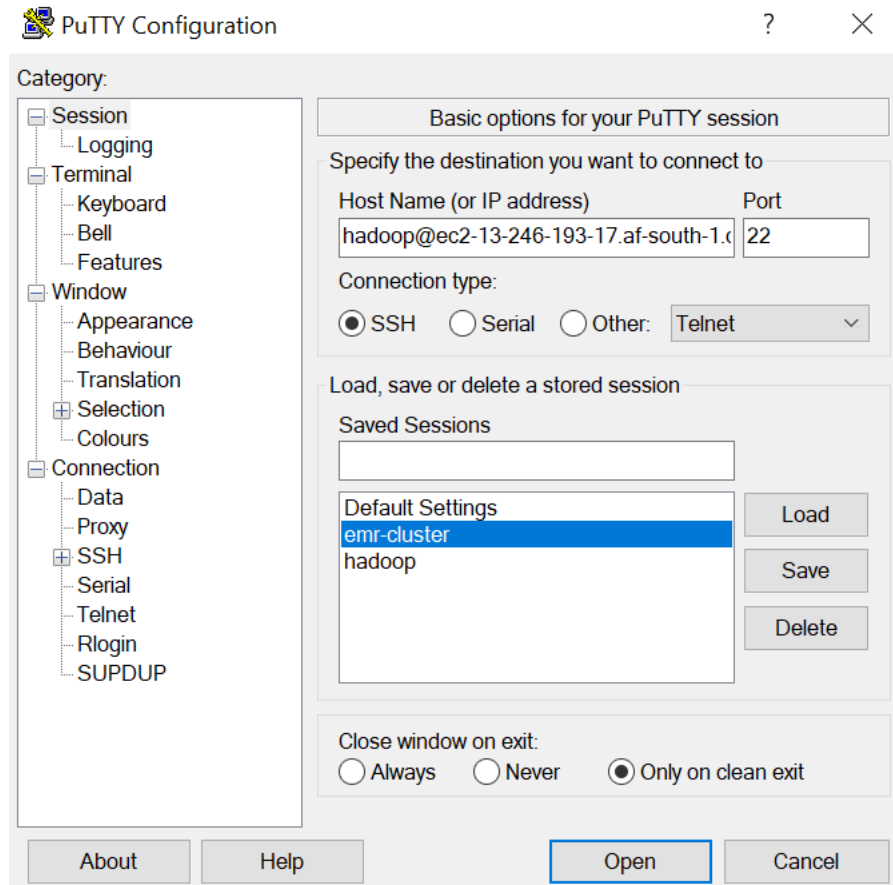


Figure 4 : Paramétrage de la session PuTTY « emr-cluster » (nom d'hôte `hadoop@ec2-...` et port 22)

2.3 Création et démarrage du cluster EMR

Le cluster « mon-cluster-mobilemoney-ci-emr » a été créé via la console AWS EMR, en version `emr-6.15.0`, avec les applications suivantes préinstallées et gérées automatiquement par AWS: Hadoop 3.3.6, Hive 3.1.3, Spark 3.4.1, Livy 0.7.1 et JupyterEnterpriseGateway.

- Configuration : groupes d'instances, 1 nœud primaire + 1 nœud principal + 1 nœud de tâche
- Région : Afrique (Le Cap) : `af-south-1`
- Destination des journaux : bucket S3 configuré à l'étape précédente

Rapport de déploiement et d'exploitation d'un Cluster Amazon EMR

The screenshot shows the AWS Management Console for the 'mon-cluster-mobilemoney-ci-emr' cluster. A green banner at the top indicates the cluster has been created. Below this, a warning message states that support for this EMR version will end on August 01, 2026, and recommends upgrading to the latest version. The main section, titled 'Récapitulatif', provides a summary of the cluster's configuration:

Informations sur le cluster	Applications	Gestion des clusters	Statut et heure
ID de cluster j-KPV53FZMQ4ZO ARN du cluster arn:aws:elasticmapreduce:af-south-1:966801367745:cluster/j-KPV53FZMQ4ZO Configuration de cluster Groupes d'instances Capacité	Version d'Amazon EMR emr-6.15.0 Applications installées Hadoop 3.3.6, Hive 3.1.3, JupyterEnterpriseGateway 2.6.0, Livy 0.7.1, Spark 3.4.1	Destination des journaux dans Amazon S3 aws-logs-966801367745-af-south-1/elasticmapreduce DNS public du nœud primaire - Connexion au nœud primaire à l'aide de SSM	Statut 🔄 Démarrage en cours Heure de création 2 juillet 2026 17:15 (UTC+00:00) Temps écoulé 2 secondes

Figure 5 : Cluster EMR créé avec statut « Démarrage en cours » et récapitulatif des informations générales

The screenshot shows the same AWS Management Console page, but the cluster's status has changed to 'Waiting'. The 'Récapitulatif' section now includes additional details about the persistent application interfaces:

Informations sur le cluster	Applications	Gestion des clusters	Statut et heure
ID de cluster j-KPV53FZMQ4ZO ARN du cluster arn:aws:elasticmapreduce:af-south-1:966801367745:cluster/j-KPV53FZMQ4ZO Configuration de cluster Groupes d'instances Capacité 1 primaire(s) 1 unité(s) principale(s) 1 tâche(s)	Version d'Amazon EMR emr-6.15.0 Applications installées Hadoop 3.3.6, Hive 3.1.3, JupyterEnterpriseGateway 2.6.0, Livy 0.7.1, Spark 3.4.1	Destination des journaux dans Amazon S3 aws-logs-966801367745-af-south-1/elasticmapreduce Interfaces utilisateur d'application persistantes 🔗 Serveur d'historique Spark 🔗 Serveur de chronologie YARN 🔗 Interface utilisateur Tez DNS public du nœud primaire	Statut ✅ En attente Heure de création 2 juillet 2026 17:15 (UTC+00:00) Temps écoulé 6 minutes, 27 secondes

Figure 6 : Cluster EMR passé au statut « En attente »

3. Exploitation du cluster EMR

3.1 Vérification des services Hadoop, YARN et Spark

La commande `jps -l` exécutée sur le nœud maître confirme le bon fonctionnement de l'ensemble des services managés par EMR, sans aucune configuration manuelle requise : NameNode (HDFS), ResourceManager (YARN), Spark History Server, Livy Server, Zookeeper, JobHistoryServer, ApplicationHistoryServer, entre autres.

```
[hadoop@ip-172-31-16-233 ~]$ sudo jps -l
24512 org.apache.hadoop.hdfs.server.namenode.NameNode
5474 aws157.logpusher.Main
27874 org.apache.hadoop.yarn.server.resourcemanager.ResourceManager
20772 com.amazonaws.emr.http.AgentHttpServer
3237 sun.tools.jps.Jps
27686 org.apache.hadoop.yarn.server.applicationhistoryservice.ApplicationHistoryServer
5803 aws157.instancecontroller.Main
331 org.apache.hadoop.util.RunJar
27409 org.apache.spark.deploy.history.HistoryServer
5650 aws157.apppusher.Main
31986 org.apache.hadoop.util.RunJar
2355 emr.metricscollector.Main
28116 org.apache.hadoop.mapreduce.v2.hs.JobHistoryServer
22997 org.apache.catalina.startup.Bootstrap
25845 org.apache.hadoop.crypto.key.kms.server.KMSWebServer
25336 org.apache.hadoop.yarn.server.webproxy.WebAppProxyServer
27002 org.apache.livy.server.LivyServer
3291 -- process information unavailable
26655 org.apache.zookeeper.server.quorum.QuorumPeerMain
```

Figure 9 : Sortie de la commande « `jps -l` » listant les processus Hadoop/YARN/Spark actifs sur le nœud maître

Cette étape illustre l'avantage principal d'EMR par rapport à l'installation manuelle sur EC2 : l'ensemble des démons Hadoop/Spark est déployé, configuré et démarré automatiquement par AWS.

3.2 Architecture du Data Lake sur HDFS

La même architecture en trois zones que sur l'installation EC2 a été reproduite sur le HDFS du cluster EMR, afin de conserver une cohérence méthodologique entre les deux approches :

- `/datalake/raw` : données brutes (CSV source)
- `/datalake/processed` : données transformées au format Parquet, partitionnées par opérateur
- `/datalake/curated` : résultats des analyses métier, prêts à la consommation

Rapport de déploiement et d'exploitation d'un Cluster Amazon EMR

```
[hadoop@ip-172-31-16-233 ~]$ hdfs dfs -mkdir -p /datalake/raw
[hadoop@ip-172-31-16-233 ~]$ hdfs dfs -mkdir -p /datalake/processed
[hadoop@ip-172-31-16-233 ~]$ hdfs dfs -mkdir -p /datalake/curated
[hadoop@ip-172-31-16-233 ~]$
[hadoop@ip-172-31-16-233 ~]$ # Vérification
[hadoop@ip-172-31-16-233 ~]$ hdfs dfs -ls /datalake
Found 3 items
drwxr-xr-x - hadoop hdfsadmingroup 0 2026-07-02 17:37 /datalake/curated
drwxr-xr-x - hadoop hdfsadmingroup 0 2026-07-02 17:37 /datalake/processed
drwxr-xr-x - hadoop hdfsadmingroup 0 2026-07-02 17:37 /datalake/raw
[hadoop@ip-172-31-16-233 ~]$
```

Figure 10 : Création des zones Raw / Processed / Curated sur HDFS et vérification de leur présence

3.3 Ingestion des données depuis Amazon S3

Le fichier transactions_mobile_money_100k.csv (13,85 Mo, 100 000 lignes) a été récupéré depuis le bucket S3 datalake-mobilemoney-ci-ms via la commande aws s3 cp, avant d'être chargé dans la zone Raw du HDFS du cluster.

```
[hadoop@ip-172-31-16-233 ~]$ aws s3 cp s3://datalake-mobilemoney-ci-ms/raw/transactions_mobile_money_100k.csv ~/transactions_mobile_money_100k.csv
download: s3://datalake-mobilemoney-ci-ms/raw/transactions_mobile_money_100k.csv to ./transactions_mobile_money_100k.csv
[hadoop@ip-172-31-16-233 ~]$ hdfs dfs -put ~/transactions_mobile_money_100k.csv /datalake/raw/
[hadoop@ip-172-31-16-233 ~]$ hdfs dfs -ls /datalake/raw/
Found 1 items
-rw-r--r-- 1 hadoop hdfsadmingroup 13854541 2026-07-02 17:40 /datalake/raw/transactions_mobile_money_100k.csv
[hadoop@ip-172-31-16-233 ~]$
```

Figure 11 : Copie du dataset depuis S3 vers le nœud maître, puis ingestion dans /datalake/raw/ sur HDFS

4. Traitement ETL avec Spark

4.1 Script de conversion CSV vers Parquet

Un script PySpark (etl_mobilemoney.py) a été créé directement sur le nœud maître afin de lire le CSV depuis la zone Raw, puis d'écrire les données au format Parquet dans la zone Processed, partitionnées par colonne operateur. Contrairement à l'exécution locale sur EC2 (--master local[2]), l'exécution ici utilise --master yarn, ce qui délègue la distribution des tâches au gestionnaire de ressources du cluster.

Rapport de déploiement et d'exploitation d'un Cluster Amazon EMR

```
[hadoop@ip-172-31-16-233 ~]$ cat > ~/etl_mobilemoney.py << 'EOF'
> from pyspark.sql import SparkSession
>
> spark = SparkSession.builder \
>     .appName("ETL-MobileMoney-CSV-to-Parquet") \
>     .getOrCreate()
>
> # Lecture du CSV depuis HDFS zone Raw
> df = spark.read.option("header", "true") \
>     .option("inferSchema", "true") \
>     .csv("hdfs:///datalake/raw/transactions_mobile_money_100k.csv")
>
> print(f"Nombre de lignes : {df.count()}")
> df.printSchema()
>
> # Écriture Parquet partitionnée dans HDFS zone Processed
> df.write.mode("overwrite") \
>     .partitionBy("opérateur") \
>     .parquet("hdfs:///datalake/processed/")
>
> print("ETL terminé avec succès !")
> spark.stop()
> EOF
[hadoop@ip-172-31-16-233 ~]$ cat ~/etl_mobilemoney.py
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("ETL-MobileMoney-CSV-to-Parquet") \
    .getOrCreate()

# Lecture du CSV depuis HDFS zone Raw
df = spark.read.option("header", "true") \
    .option("inferSchema", "true") \
    .csv("hdfs:///datalake/raw/transactions_mobile_money_100k.csv")

print(f"Nombre de lignes : {df.count()}")
df.printSchema()

# Écriture Parquet partitionnée dans HDFS zone Processed
df.write.mode("overwrite") \
    .partitionBy("opérateur") \
    .parquet("hdfs:///datalake/processed/")

print("ETL terminé avec succès !")
spark.stop()
[hadoop@ip-172-31-16-233 ~]$
```

Figure 12 : Création du script `etl_mobilemoney.py` (lecture CSV, écriture Parquet partitionnée) sur le nœud maître

4.2 Vérification des résultats de l'ETL

Après exécution via `spark-submit --master yarn`, la zone `/datalake/processed/` contient bien les quatre partitions attendues, une par opérateur (MTN CI, Moov Africa, Orange Money, Wave), chacune constituée de plusieurs fichiers Parquet signe du parallélisme réel apporté par les multiples exécuteurs Spark du cluster, contrairement à l'exécution mono-nœud sur EC2 qui produisait un seul fichier par partition.

```
[hadoop@ip-172-31-16-233 ~]$ hdfs dfs -ls /datalake/processed/
Found 5 items
-rw-r--r-- 1 hadoop hdfsadmin group 0 2026-07-02 17:45 /datalake/processed/_SUCCESS
drwxr-xr-x - hadoop hdfsadmin group 0 2026-07-02 17:45 /datalake/processed/opérateur=MTN CI
drwxr-xr-x - hadoop hdfsadmin group 0 2026-07-02 17:45 /datalake/processed/opérateur=Moov Africa
drwxr-xr-x - hadoop hdfsadmin group 0 2026-07-02 17:45 /datalake/processed/opérateur=Orange Money
drwxr-xr-x - hadoop hdfsadmin group 0 2026-07-02 17:45 /datalake/processed/opérateur=Wave
[hadoop@ip-172-31-16-233 ~]$
```

Figure 13 : Vérification de la zone `/datalake/processed/` avec 4 partitions opérateur créées avec succès

5. Analyses Spark SQL

5.1 Script d'analyse métier

Un second script (`analyse_mobilemoney.py`) reproduit les trois requêtes métier définies dans le rapport initial : volume de transactions par opérateur, répartition des statuts par opérateur, et top 5 des zones expéditrices. Les résultats de chaque requête sont affichés à l'écran puis sauvegardés au format CSV dans la zone Curated du Data Lake.

```
hadoop@ip-172-31-16-233:~$ cat > ~/analyse_mobilemoney.py << 'EOF'
> from pyspark.sql import SparkSession
>
> spark = SparkSession.builder \
>   .appName("Analyse-MobileMoney") \
>   .getOrCreate()
>
> df = spark.read.parquet("hdfs:///datalake/processed/")
> df.createOrReplaceTempView("transactions")
>
> # Requête 1 : Volume par opérateur
> req1 = spark.sql("""
>   SELECT operateur,
>          COUNT(*) AS nb_transactions,
>          ROUND(SUM(montant_fcfa), 0) AS volume_total_fcfa
>   FROM transactions
>   GROUP BY operateur
>   ORDER BY nb_transactions DESC
> """)
> req1.show()
> req1.write.mode("overwrite").csv("hdfs:///datalake/curated/req1_volume_operateur", header=True)
>
> # Requête 2 : Taux de succès par opérateur
> req2 = spark.sql("""
>   SELECT operateur, statut, COUNT(*) AS nb
>   FROM transactions
>   GROUP BY operateur, statut
>   ORDER BY operateur, statut
> """)
> req2.show(20)
> req2.write.mode("overwrite").csv("hdfs:///datalake/curated/req2_statut_operateur", header=True)
>
> # Requête 3 : Top 5 zones expéditrices
> req3 = spark.sql("""
>   SELECT zone_expéditeur, COUNT(*) AS nb_transactions
>   FROM transactions
>   GROUP BY zone_expéditeur
>   ORDER BY nb_transactions DESC
>   LIMIT 5
> """)
> req3.show()
> req3.write.mode("overwrite").csv("hdfs:///datalake/curated/req3_top5_zones", header=True)
>
> print("Analyses terminées avec succès !")
> spark.stop()
> EOF
[hadoop@ip-172-31-16-233 ~]$ spark-submit --master yarn ~/analyse_mobilemoney.py
```

Figure 14 : Création et lancement du script `analyse_mobilemoney.py` avec 3 requêtes Spark SQL)

5.2 Résultats obtenus

Requête 1 : Répartition des statuts par opérateur : chaque opérateur affiche un taux de succès homogène d'environ 80 %, avec environ 10 % de transactions en attente et 10 % en échec, une distribution cohérente et stable entre les quatre opérateurs.

operateur	statut	nb
MTN CI	En attente	2448
MTN CI	Succès	19817
MTN CI	Échec	2428
Moov Africa	En attente	2495
Moov Africa	Succès	20072
Moov Africa	Échec	2521
Orange Money	En attente	2496
Orange Money	Succès	20100
Orange Money	Échec	2501
Wave	En attente	2516
Wave	Succès	20171
Wave	Échec	2435

Figure 15 : Résultat de la requête « statut par opérateur » (répartition En attente / Succès / Échec)

Requête 2 : Volume de transactions par opérateur : Wave arrive en tête avec 25 122 transactions ($\approx 4,64$ milliards FCFA), suivi de près par Orange Money, Moov Africa et MTN CI. Les volumes entre opérateurs sont très équilibrés, avec un écart inférieur à 2 %.

operateur	nb_transactions	volume_total_fcfa
Wave	25122	4643339719
Orange Money	25097	4633409970
Moov Africa	25088	4636951482
MTN CI	24693	4551368564

Figure 16 : Résultat de la requête « volume de transactions par opérateur » (nombre de transactions et volume en FCFA)

Ces résultats, comme pour l'installation EC2, proviennent d'un dataset simulé à distribution quasi uniforme. Il convient donc d'éviter toute conclusion trop affirmative sur une hiérarchie commerciale réelle entre opérateurs ou zones géographiques à partir de ces seuls chiffres.

6. Supervision et preuve du parallélisme distribué

6.1 Journaux d'exécution Spark

Les journaux détaillés de l'exécution (spark-submit --master yarn) confirment la répartition des tâches entre les différents exécuteurs du cluster. On y observe notamment l'attribution de tâches à l'« executor 2 » sur le nœud ip-172-31-19-197, distinct du nœud maître, ainsi que la confirmation finale « Analyses terminées avec succès ».

```
hadoop@ip-172-31-16-233:~$
26/07/02 18:05:55 INFO SparkContext: Starting job: csv at NativeMethodAccessorImpl.java:0
26/07/02 18:05:55 INFO DAGScheduler: Got job 16 (csv at NativeMethodAccessorImpl.java:0) with 1 output partitions
26/07/02 18:05:55 INFO DAGScheduler: Final stage: ResultStage 28 (csv at NativeMethodAccessorImpl.java:0)
26/07/02 18:05:55 INFO DAGScheduler: Parents of final stage: List(ShuffleMapStage 27)
26/07/02 18:05:55 INFO DAGScheduler: Missing parents: List()
26/07/02 18:05:55 INFO DAGScheduler: Submitting ResultStage 28 (MapPartitionsRDD[59] at csv at NativeMethodAccessorImpl.java:0), which has no missing parents
26/07/02 18:05:55 INFO MemoryStore: Block broadcast_28 stored as values in memory (estimated size 285.2 KiB, free 905.9 MiB)
26/07/02 18:05:55 INFO MemoryStore: Block broadcast_28_piece0 stored as bytes in memory (estimated size 109.7 KiB, free 905.8 MiB)
26/07/02 18:05:55 INFO BlockManagerInfo: Added broadcast_28_piece0 in memory on ip-172-31-16-233.af-south-1.compute.internal:36945 (size: 109.7 KiB, free: 911.4 MiB)
26/07/02 18:05:55 INFO SparkContext: Created broadcast 28 from broadcast at DAGScheduler.scala:1592
26/07/02 18:05:55 INFO DAGScheduler: Submitting 1 missing tasks from ResultStage 28 (MapPartitionsRDD[59] at csv at NativeMethodAccessorImpl.java:0) (first 15 tasks are for partitions Vector(0))
26/07/02 18:05:55 INFO YarnScheduler: Adding task set 28.0 with 1 tasks resource profile 0
26/07/02 18:05:55 INFO TaskSetManager: Starting task 0.0 in stage 28.0 (TID 58) (ip-172-31-19-197.af-south-1.compute.internal, executor 2, partition 0, PROCESS_LOCAL, 7374 bytes)
26/07/02 18:05:55 INFO BlockManagerInfo: Added broadcast_28_piece0 in memory on ip-172-31-19-197.af-south-1.compute.internal:38049 (size: 109.7 KiB, free: 4.8 GiB)
26/07/02 18:05:55 INFO MapOutputTrackerMasterEndpoint: Asked to send map output locations for shuffle 7 to 172.31.19.197:50826
26/07/02 18:05:55 INFO TaskSetManager: Finished task 0.0 in stage 28.0 (TID 58) in 101 ms on ip-172-31-19-197.af-south-1.compute.internal (executor 2) (1/1)
26/07/02 18:05:55 INFO YarnScheduler: Removed TaskSet 28.0, whose tasks have all completed, from pool
26/07/02 18:05:55 INFO DAGScheduler: ResultStage 28 (csv at NativeMethodAccessorImpl.java:0) finished in 0.149 s
26/07/02 18:05:55 INFO DAGScheduler: Job 16 is finished. Cancelling potential speculative or zombie tasks for this job
26/07/02 18:05:55 INFO YarnScheduler: Killing all running tasks in stage 28: Stage finished
26/07/02 18:05:55 INFO DAGScheduler: Job 16 finished: csv at NativeMethodAccessorImpl.java:0, took 0.154082 s
26/07/02 18:05:55 INFO FileFormatWriter: Start to commit write Job 26242de6-7f46-4cb0-b8da-b0c8482e6eeb.
26/07/02 18:05:55 INFO FileFormatWriter: Write Job 26242de6-7f46-4cb0-b8da-b0c8482e6eeb committed. Elapsed time: 15 ms.
26/07/02 18:05:55 INFO FileFormatWriter: Finished processing stats for write job 26242de6-7f46-4cb0-b8da-b0c8482e6eeb.
Analyses terminées avec succès !
26/07/02 18:05:55 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/07/02 18:05:55 INFO SparkUI: Stopped Spark web UI at http://ip-172-31-16-233.af-south-1.compute.internal:4040
26/07/02 18:05:55 INFO YarnClientSchedulerBackend: Interrupting monitor thread
26/07/02 18:05:55 INFO YarnClientSchedulerBackend: Shutting down all executors
26/07/02 18:05:55 INFO YarnClientSchedulerBackend: Asking each executor to shut down
26/07/02 18:05:55 INFO YarnClientSchedulerBackend: YARN client scheduler backend Stopped
26/07/02 18:05:55 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/07/02 18:05:55 INFO MemoryStore: MemoryStore cleared
26/07/02 18:05:55 INFO BlockManager: BlockManager stopped
26/07/02 18:05:55 INFO BlockManagerMaster: BlockManagerMaster stopped
26/07/02 18:05:55 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/07/02 18:05:55 INFO SparkContext: Successfully stopped SparkContext
26/07/02 18:05:56 INFO ShutdownHookManager: Shutdown hook called
26/07/02 18:05:56 INFO ShutdownHookManager: Deleting directory /mnt/tmp/spark-552370c4-499a-4b54-b8b1-d38d1f14272e/pyspark-a53d3dee-2f95-4460-aae9-f90a7a955ec
26/07/02 18:05:56 INFO ShutdownHookManager: Deleting directory /mnt/tmp/spark-6369f144-1225-40b5-b6c1-edc7690bb396
26/07/02 18:05:56 INFO ShutdownHookManager: Deleting directory /mnt/tmp/spark-552370c4-499a-4b54-b8b1-d38d1f14272e
hadoop@ip-172-31-16-233:~$
```

Figure 17 : Extrait des journaux Spark montrant l'exécution distribuée sur l'executor 2 et la confirmation de fin de traitement

6.2 Vérification finale de la zone Curated

La commande `hdfs dfs -ls -R` confirme la présence des trois jeux de résultats dans la zone Curated, chacun accompagné d'un marqueur `_SUCCESS` attestant de la réussite de l'écriture.

```
hadoop@ip-172-31-16-233:~$ hdfs dfs -ls -R /datalake/curated/
drwxr-xr-x - hadoop hdfsadmin group 0 2026-07-02 18:05 /datalake/curated/req1_volume_operateur
-rw-r--r-- 1 hadoop hdfsadmin group 0 2026-07-02 18:05 /datalake/curated/req1_volume_operateur/_SUCCESS
-rw-r--r-- 1 hadoop hdfsadmin group 149 2026-07-02 18:05 /datalake/curated/req1_volume_operateur/part-00000-b75fac37-ef5b-4f36-adb1-2c12a3e48d2e-c000.csv
drwxr-xr-x - hadoop hdfsadmin group 0 2026-07-02 18:05 /datalake/curated/req2_statut_operateur
-rw-r--r-- 1 hadoop hdfsadmin group 0 2026-07-02 18:05 /datalake/curated/req2_statut_operateur/_SUCCESS
-rw-r--r-- 1 hadoop hdfsadmin group 299 2026-07-02 18:05 /datalake/curated/req2_statut_operateur/part-00000-f7d45d21-3805-42f8-b698-b9e302802d0c-c000.csv
drwxr-xr-x - hadoop hdfsadmin group 0 2026-07-02 18:05 /datalake/curated/req3_top5_zones
-rw-r--r-- 1 hadoop hdfsadmin group 0 2026-07-02 18:05 /datalake/curated/req3_top5_zones/_SUCCESS
-rw-r--r-- 1 hadoop hdfsadmin group 115 2026-07-02 18:05 /datalake/curated/req3_top5_zones/part-00000-169d69b0-4619-4ce3-bd13-c644f0b5b301-c000.csv
hadoop@ip-172-31-16-233:~$
```

Figure 18 : Contenu final de la zone /datalake/curated/ montrant les 3 répertoires de résultats avec fichiers `_SUCCESS` et CSV

6.3 Interfaces de supervision du cluster

Le cluster EMR expose nativement des interfaces web de supervision (YARN ResourceManager, Spark History Server, Tez, Serveur de chronologie YARN), accessibles soit en direct depuis le cluster, soit via des interfaces persistantes disponibles 30 jours après l'arrêt du cluster. Ces interfaces ont permis de documenter concrètement l'exécution distribuée du pipeline.

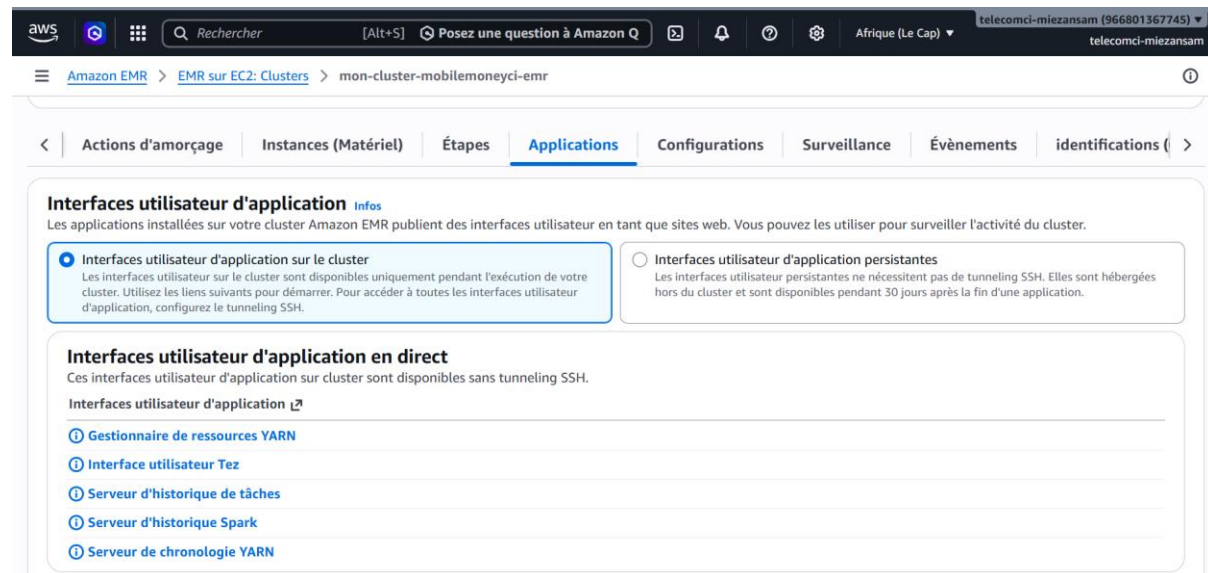
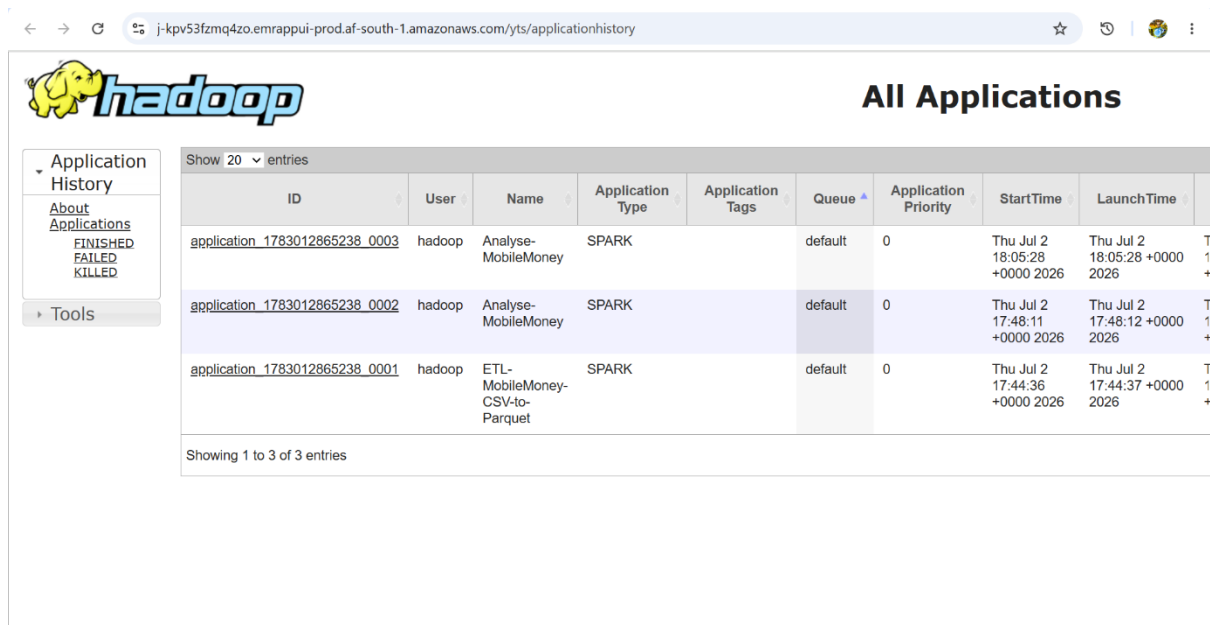


Figure 19 : Page « Applications » du cluster EMR listant les interfaces utilisateur disponibles (YARN, Spark History Server, Tez)

La page « All Applications » du serveur de chronologie YARN liste les trois jobs Spark exécutés sur le cluster (l'ETL de conversion CSV vers Parquet, puis les deux exécutions du script d'analyse), confirmant leur bonne prise en charge par le gestionnaire de ressources.

Rapport de déploiement et d'exploitation d'un Cluster Amazon EMR

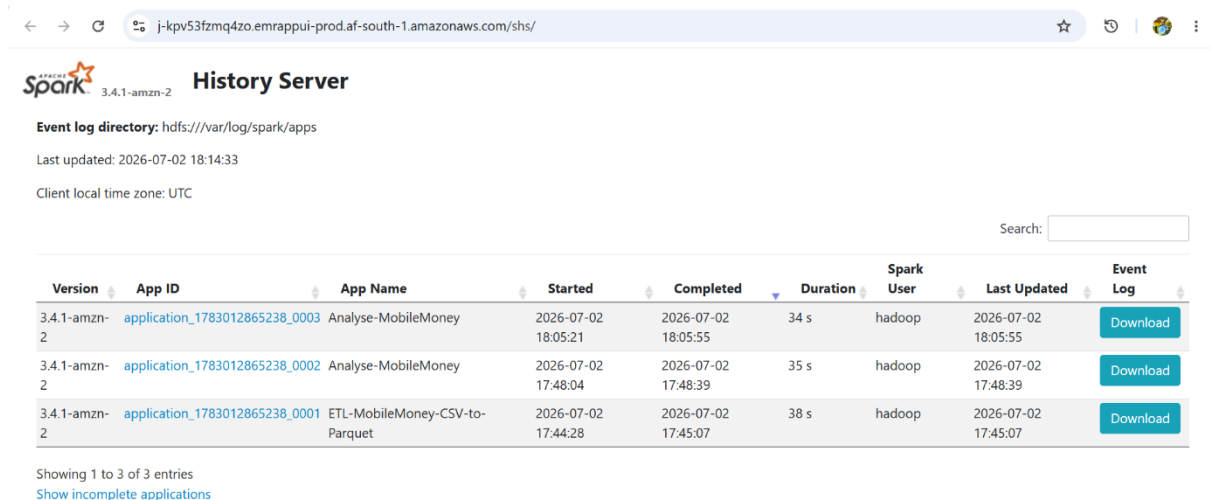


ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	
application_1783012865238_0003	hadoop	Analyse-MobileMoney	SPARK		default	0	Thu Jul 2 18:05:28 +0000 2026	Thu Jul 2 18:05:28 +0000 2026	T 1' +
application_1783012865238_0002	hadoop	Analyse-MobileMoney	SPARK		default	0	Thu Jul 2 17:48:11 +0000 2026	Thu Jul 2 17:48:12 +0000 2026	T 1' +
application_1783012865238_0001	hadoop	ETL-MobileMoney-CSV-to-Parquet	SPARK		default	0	Thu Jul 2 17:44:36 +0000 2026	Thu Jul 2 17:44:37 +0000 2026	T 1' +

Showing 1 to 3 of 3 entries

Figure 20 : YARN Application History listant des 3 applications Spark exécutées (ETL-MobileMoney-CSV-to-Parquet, Analyse-MobileMoney x2)

Le Spark History Server fournit une vue synthétique des trois jobs avec leur durée réelle : 38 secondes pour l'ETL, 35 puis 34 secondes pour les deux exécutions de l'analyse. Ces chiffres, mesurés directement par Spark, sont plus fiables qu'une estimation manuelle.



Version	App ID	App Name	Started	Completed	Duration	Spark User	Last Updated	Event Log
3.4.1-amzn-2	application_1783012865238_0003	Analyse-MobileMoney	2026-07-02 18:05:21	2026-07-02 18:05:55	34 s	hadoop	2026-07-02 18:05:55	Download
3.4.1-amzn-2	application_1783012865238_0002	Analyse-MobileMoney	2026-07-02 17:48:04	2026-07-02 17:48:39	35 s	hadoop	2026-07-02 17:48:39	Download
3.4.1-amzn-2	application_1783012865238_0001	ETL-MobileMoney-CSV-to-Parquet	2026-07-02 17:44:28	2026-07-02 17:45:07	38 s	hadoop	2026-07-02 17:45:07	Download

Showing 1 to 3 of 3 entries
[Show incomplete applications](#)

Figure 21 : Server montrant le tableau récapitulatif des 3 jobs avec leur durée d'exécution

La page « Cluster Metrics » du YARN ResourceManager confirme la présence de 2 nœuds actifs (Active Nodes) sur le cluster, ainsi que les 3 applications soumises et complétées avec succès, sans aucun échec.

Rapport de déploiement et d'exploitation d'un Cluster Amazon EMR

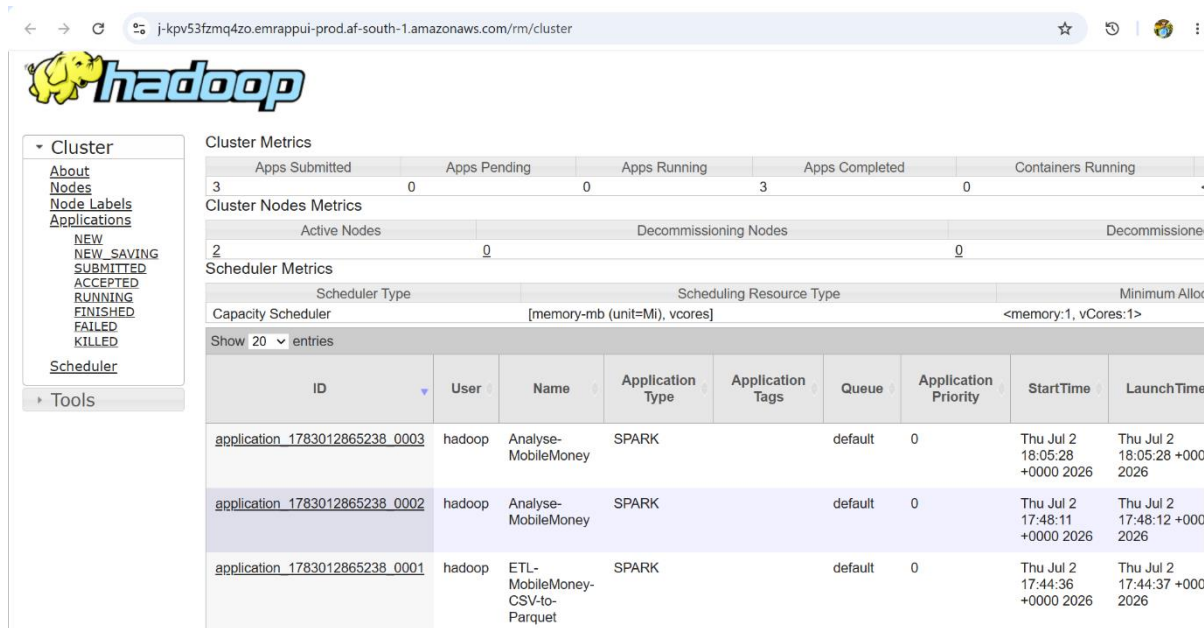


Figure 22 : Cluster Metrics : 2 Active Nodes, 3 Apps Completed

Le détail de l'application application_..._0003 (le second job d'analyse) confirme un état final FINISHED/SUCCEDED, pour une durée totale de 27 secondes, avec le conteneur applicatif hébergé sur un nœud distinct du nœud maître (ip-172-31-19-197).

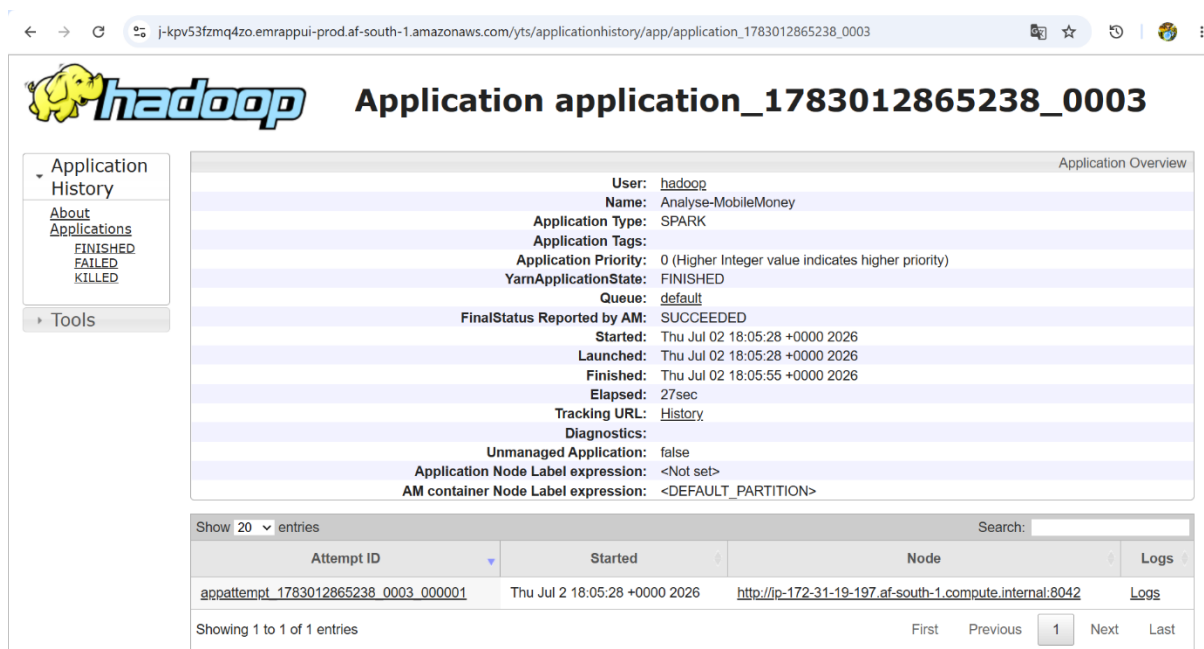


Figure 23 : Détail de l'application YARN application_..._0003 avec statut FINISHED/SUCCEDED, durée 27 sec

L'onglet « Jobs » du Spark History Server montre que le script d'analyse a été décomposé en 17 jobs internes par le moteur Spark (un par action déclenchée : les trois requêtes SQL, leurs écritures CSV et leurs affichages), chacun avec son propre suivi de stages et de tâches.

Rapport de déploiement et d'exploitation d'un Cluster Amazon EMR

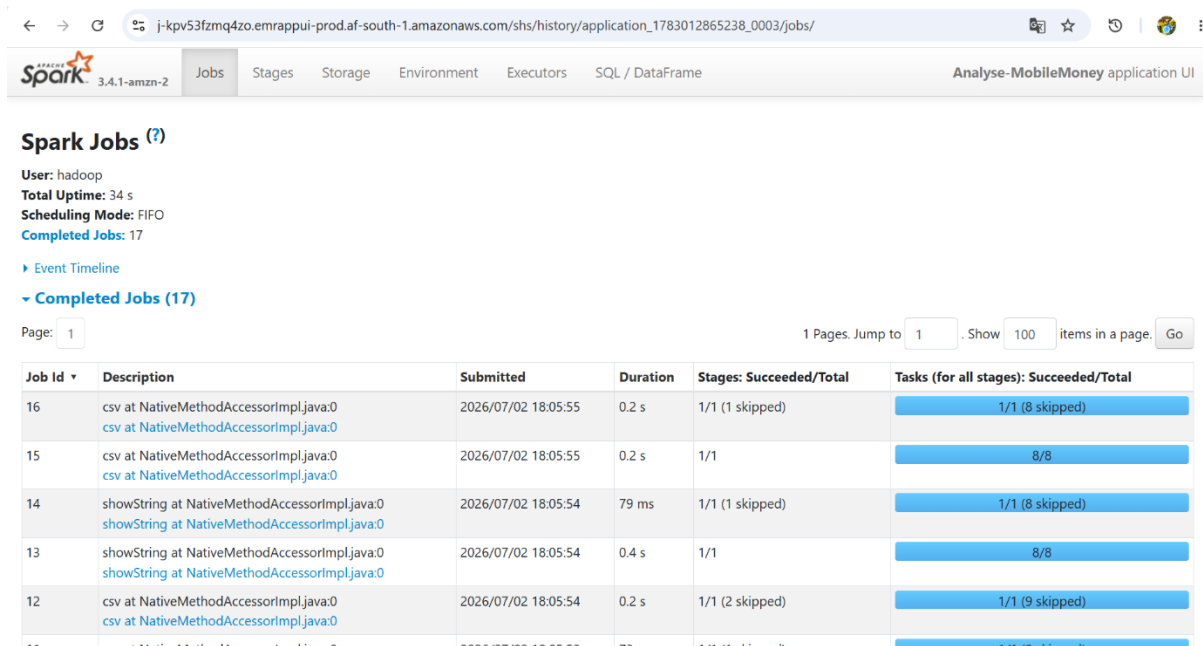


Figure 24 : Détail des 17 jobs internes de l'application Analyse-MobileMoney

L'Event Timeline du job Spark montre l'ajout séquentiel du driver puis de deux exécuteurs distincts (Executor 1 et Executor 2), démontrant que Spark a bien alloué plusieurs exécuteurs sur le cluster pour traiter les données en parallèle, à la différence de l'exécution mono-nœud sur EC2.

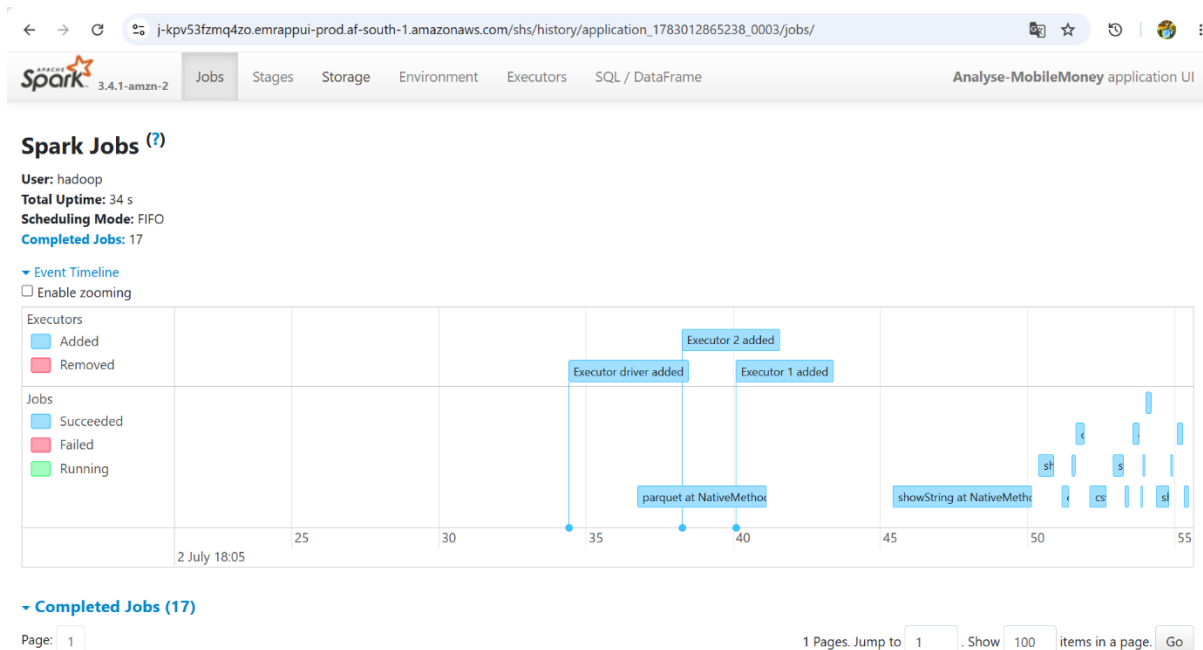


Figure 25 : Ajout du driver et de 2 exécuteurs distincts (Executor 1, Executor 2)

Le tableau des exécuteurs confirme 3 exécuteurs actifs (driver + 2 exécuteurs de calcul), 8 cœurs au total, et 59 tâches complétées sans aucun échec. Le temps de calcul cumulé (1,2 min) supérieur au temps réel écoulé (34 s) démontre que plusieurs tâches se sont bien exécutées simultanément sur des cœurs différents.

Rapport de déploiement et d'exploitation d'un Cluster Amazon EMR

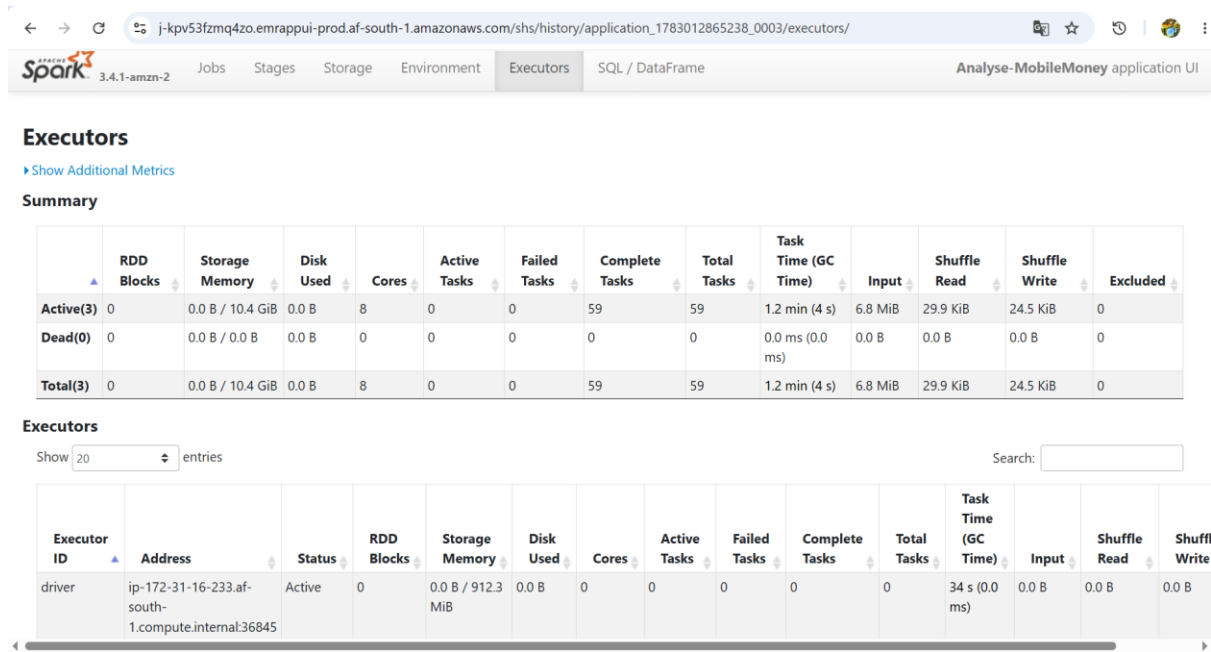


Figure 26 : 3 exécuteurs actifs, 8 cœurs, 59 tâches complétées, 1,2 min de temps de calcul cumulé

Rapport de déploiement et d'exploitation d'un Cluster Amazon EMR

7. Comparaison EC2 manuel vs EMR managé

Critère	Installation EC2 manuelle	Cluster Amazon EMR
Installation Hadoop/Spark	Manuelle (téléchargement, configuration XML)	Automatique, prête à l'emploi
Nombre de nœuds	1 (mode pseudo-distribué)	Plusieurs (cluster réellement distribué)
Gestion mémoire / OOM	À gérer manuellement	Gérée automatiquement par AWS
Accès IAM à S3	Configuration manuelle des clés	Rôle IAM attribué automatiquement
Scalabilité	Aucune	Ajout / retrait de nœuds à la demande
Mode d'exécution Spark	--master local[2]	--master yarn
Fichiers Parquet par partition	1 seul fichier (mono-nœud)	Plusieurs fichiers (écriture parallèle multi-exécuteurs)

8. Conclusion

Le déploiement du cluster Amazon EMR a permis de reproduire à l'identique le pipeline de traitement Big Data conçu initialement sur une instance EC2 unique, tout en démontrant concrètement les bénéfices d'une infrastructure managée et distribuée : automatisation du déploiement, répartition réelle des tâches sur plusieurs nœuds, et disponibilité d'interfaces de supervision intégrées (YARN, Spark History Server). Les résultats analytiques obtenus sont cohérents avec ceux du rapport EC2, ce qui valide la reproductibilité du pipeline ETL et des analyses Spark SQL indépendamment de l'infrastructure sous-jacente. Cette expérimentation constitue ainsi une démonstration concrète de la perspective d'évolution évoquée dans le rapport initial : le passage d'une architecture pseudo-distribuée à un cluster multi-nœuds managé.